

Matrix Multiplication (C Programming)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10 + 40 + 90 & 11 + 42 + 93 \\ 40 + 100 + 180 & 44 + 105 + 186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

Matrix A

$$\begin{bmatrix} 3 & 2 & 1 & 5 \\ 9 & 1 & 3 & 0 \end{bmatrix}$$

4 columns

Matrix B

$$\begin{bmatrix} 2 & 9 & 0 \\ 1 & 3 & 5 \\ 2 & 4 & 7 \\ 8 & 1 & 5 \end{bmatrix}$$

4 rows

4 columns = 4 rows

© mathwarehouse.com

1. Introduction

Matrix multiplication is a mathematical operation where **two matrices are multiplied to produce a new matrix**.

However, matrix multiplication is possible only when a specific condition is satisfied.

Condition for Multiplication

If matrix **A** is:

$$A = m \times n$$

and matrix **B** is:

$$B = n \times p$$

Then multiplication is possible and the result matrix **C** will be:

$$C = m \times p$$

Meaning:

Number of columns of A = Number of rows of B

2. Example

Matrix **A** (2×3)

```
1 2 3
4 5 6
```

Matrix **B** (3×2)

```
7 8
9 10
11 12
```

Result Matrix **C** (2×2)

58 64
139 154

Calculation Example

First element:

$$(1 \times 7) + (2 \times 9) + (3 \times 11) = 58$$

3. Matrix Multiplication Algorithm

Steps:

1. Input matrix **A** and **B**.
2. Check if multiplication is possible.
3. Use **three nested loops**.
4. Multiply rows of A with columns of B.
5. Add the products.
6. Store result in matrix **C**.

Formula:

$$C[i][j] = \sum A[i][k] \times B[k][j]$$

4. C Program for Matrix Multiplication

```
#include<stdio.h>
```

```
int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2, i, j, k;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d",&r1,&c1);

    printf("Enter rows and columns of Matrix B: ");
```

```

scanf("%d %d",&r2,&c2);

if(c1 != r2)
{
    printf("Matrix multiplication not possible");
    return 0;
}

printf("Enter elements of Matrix A:\n");
for(i=0;i<r1;i++)
    for(j=0;j<c1;j++)
        scanf("%d",&A[i][j]);

printf("Enter elements of Matrix B:\n");
for(i=0;i<r2;i++)
    for(j=0;j<c2;j++)
        scanf("%d",&B[i][j]);

for(i=0;i<r1;i++)
{
    for(j=0;j<c2;j++)
    {
        C[i][j] = 0;

        for(k=0;k<c1;k++)
        {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}

printf("\nResult Matrix:\n");

for(i=0;i<r1;i++)
{
    for(j=0;j<c2;j++)
        printf("%d ",C[i][j]);

    printf("\n");
}

return 0;
}

```

5. Program Explanation

Step 1: Declare matrices

```
int A[10][10], B[10][10], C[10][10];
```

- A → first matrix
- B → second matrix
- C → result matrix

Step 2: Check multiplication condition

```
if(c1 != r2)
```

If columns of **A** ≠ rows of **B**, multiplication is impossible.

Step 3: Core multiplication logic

```
for(i=0;i<r1;i++)
{
    for(j=0;j<c2;j++)
    {
        C[i][j] = 0;

        for(k=0;k<c1;k++)
        {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Explanation:

- **i** → row of A
- **j** → column of B
- **k** → multiplication index

6. Example Execution

Input

Matrix A (2×2)

1 2
3 4

Matrix B (2×2)

5 6
7 8

Output

19 22
43 50

7. Time Complexity

Matrix multiplication requires:

$O(n^3)$

Because it uses **three nested loops**.

8. Key Points to Remember

✓ Multiplication possible only if:

columns of A = rows of B

✓ Result matrix size:

rows of A × columns of B

✓ Requires **three loops** in C.